

ECE 60126 HW1

Andrea Goh (goh8@purdue.edu)

January 2025

1 Programming Task

1.1 Task 1

A BioModel class is initialized using the `__init__` method with an input parameter named **sequence**.

```
class BioModel(object):
    def __init__(self, sequence):
        self.sequence = sequence
```

1.2 Task 2

The BioModel class is extended into a subclass called ExponentialGrowthModel, which is initialized using the `__init__` method with two input parameters: start and rate.

```
class ExponentialGrowthModel(BioModel):
    def __init__(self, start, rate):
        self.start = start
        self.rate = rate
```

1.3 Task 3

The instances of the ExponentialGrowthModel subclass are made *callable* using the `__call__` method with an input parameter named **length**. When the instance of ExponentialGrowthModel is called, a sequence of length values are generated and printed based on the formula:

$$\text{next value} = \text{current value} \times (1 + \text{rate}) \quad (1)$$

A for loop is maintained to calculate the values at each iteration using equation 1. The rounded values of each entries of the sequence are printed using a list comprehension.

```
class ExponentialGrowthModel(BioModel):
    ...
    def __call__(self, length):
        self.sequence = [self.start]
        for i in range(length-1):
            temp = self.sequence[i]*(1+self.rate)
            self.sequence.append(temp)
        print([round(elem,2) for elem in self.sequence])
        return self.sequence
```

In addition, the BioModel class is extended using the `__len__` method, which returns the length of the sequence:

```
class BioModel(object):
    ...
    def __len__(self):
        return len(self.sequence)
```

1.4 Task 4

The BioModel class is modified to be used as an iterator using the `__iter__` and `__next__` methods. The `__iter__` method initializes the index value. The `__next__` method returns the next element in the sequence as long as the index value is smaller than the length of the sequence.

```
class BioModel(object):
    ...
    def __iter__(self):
        self.index = -1
        return self
    def __next__(self):
        self.index += 1
        if self.index < len(self):
            return self.sequence[self.index]
        else:
            raise StopIteration
```

1.5 Task 5

The BioModel class is extended into a subclass called ExponentialDecayModel. This new subclass is identical to the ExponentialGrowthmodel except that it is a decay model and the rate has a negative sign:

$$\text{next value} = \text{current value} \times (1 - \text{rate}) \quad (2)$$

```
class ExponentialDecayModel(BioModel):
    def __init__(self, start, rate):
        self.start = start
        self.rate = rate
    def __call__(self, length):
        self.sequence = [self.start]
        for i in range(length-1):
            temp = self.sequence[i]*(1-self.rate)
            self.sequence.append(temp)
        print([round(elem,2) for elem in self.sequence])
        return self.sequence
```

1.6 Task 6

The BioModel class is modified using the `__eq__` method to allow for two instances to be compared using the `==` operator. An if statement is used to check that the length of the two sequence are the same. If the sequences are not the same length, a `ValueError` will be thrown. Else, a variable called **count** is initialized to 0. Then, a for loop is used to iterate through corresponding entries in both sequences, and if the entries are equal, count will increase by one.

```
class BioModel(object):
    ...
    def __eq__(self, other):
        if len(self) != len(other):
            raise ValueError('Two arrays are not equal in length!')
        else:
            count = 0
            for i in range(len(self)):
                if self.sequence[i] == other.sequence[i]:
                    count += 1
            return count
```

2 Results of Programming task

2.1 Results 1 (from assignment)

```
GM = ExponentialGrowthModel(start=100,rate=0.1)
GM(length=5) # [100, 110.0, 121.0, 133.1, 146.41]
print(len(GM)) # 5
print([round(n,2) for n in GM]) # [100, 110.0, 121.0, 133.1, 146.41]

DM = ExponentialDecayModel(start=100,rate=0.2)
DM(length=5) # [100, 80.0, 64.0, 51.2, 40.96]
print(len(DM)) # 5
print([round(n,2) for n in DM]) # [100, 80.0, 64.0, 51.2, 40.96]

GM = ExponentialGrowthModel(start=100,rate=0.1)
GM(length=5) # [100, 110.0, 121.0, 133.1, 146.41]

GM2 = ExponentialGrowthModel(start=100,rate=0.2)
GM2(length=5) # [100, 120.0, 144.0, 172.8, 207.36]

print(GM == GM2) # 1

GM3 = ExponentialGrowthModel(start=100,rate=0.2)
GM3(length=3) # [100, 120.0, 144.0]

print(GM == GM3) # ValueError: Two arrays are not equal in length!
```

2.2 Results 2 (own parameter values)

```
GM = ExponentialGrowthModel(start=50,rate=0.2)
GM(length=5) # [50, 60.0, 72.0, 86.4, 103.68]
print(len(GM)) # 5
print([round(n,2) for n in GM]) # [50, 60.0, 72.0, 86.4, 103.68]

DM = ExponentialDecayModel(start=50,rate=0.3)
DM(length=5) # [50, 35.0, 24.5, 17.15, 12.0]
print(len(DM)) # 5
print([round(n,2) for n in DM]) # [50, 35.0, 24.5, 17.15, 12.0]

GM = ExponentialGrowthModel(start=50,rate=0.2)
GM(length=5) # [50, 60.0, 72.0, 86.4, 103.68]

GM2 = ExponentialGrowthModel(start=50,rate=0.3)
GM2(length=5) # [50, 65.0, 84.5, 109.85, 142.81]

print(GM == GM2) # 1

GM3 = ExponentialGrowthModel(start=50,rate=0.2)
GM3(length=3) # [50, 60.0, 72.0]

print(GM == GM3) # ValueError: Two arrays are not equal in length!
```

3 Bonus

3.1 Combined Model

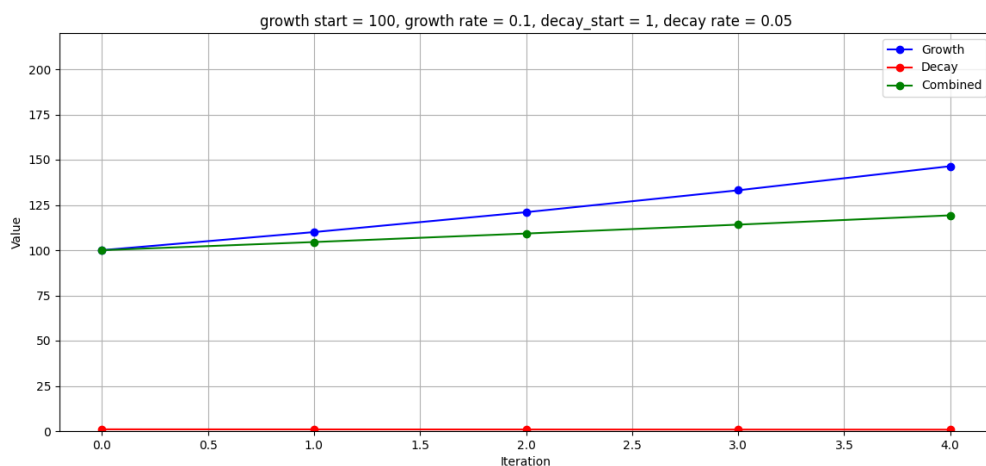
A new subclass called `CombinedBioModel` is created to combine the effects of growth and decay. The `__init__` method initializes the subclass with input parameters: `growth_start`, `growth_rate`, `decay_start`, and `decay_rate`. The instances of `CombinedBioModel` are made callable using the `__call__` method. When called, the subclass will do the following:

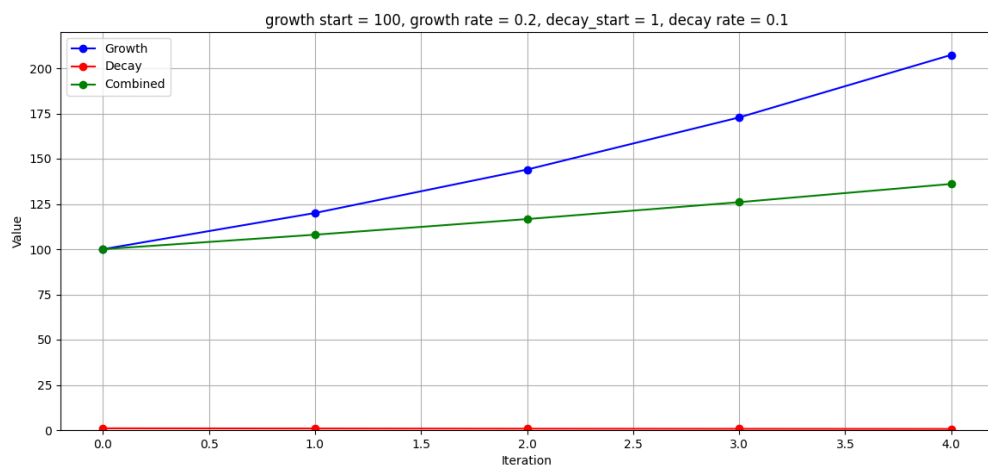
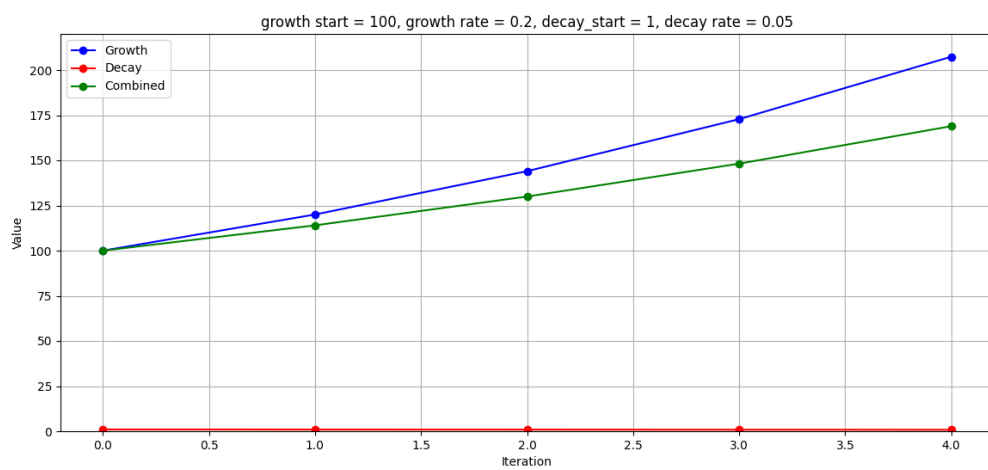
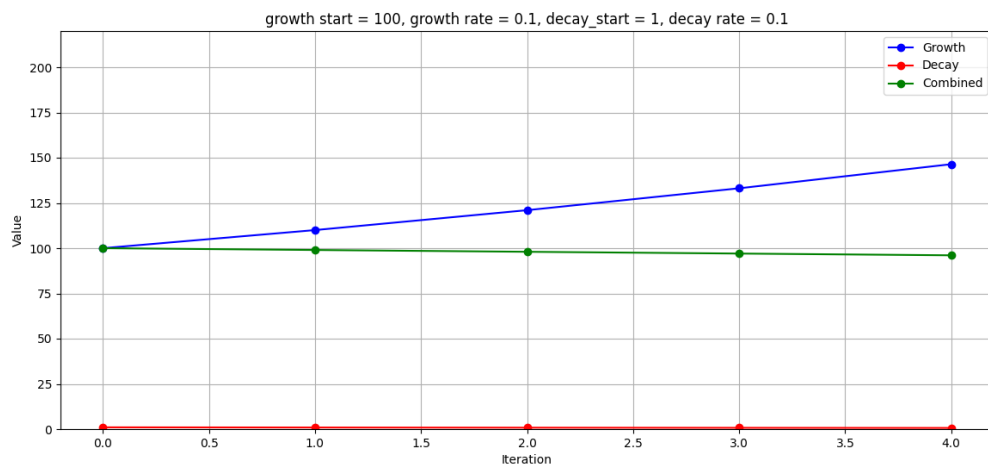
- Generate a growth sequence using `ExponentialGrowthModel`
- Generate a decay sequence using `ExponentialDecayModel`
- Combined the growth and decay sequences by multiplying values element-wise

```
class CombinedBioModel(BioModel):
    def __init__(self, growth_start, growth_rate, decay_start, decay_rate):
        self.growth_start = growth_start
        self.growth_rate = growth_rate
        self.decay_start = decay_start
        self.decay_rate = decay_rate
    def __call__(self, length):
        GM = ExponentialGrowthModel(start=self.growth_start, rate=self.growth_rate)
        DM = ExponentialDecayModel(start=self.decay_start, rate=self.decay_rate)
        self.GM_seq = GM(length)
        self.DM_seq = DM(length)
        self.cmb_seq = []
        for i in range(length):
            self.cmb_seq.append(self.GM_seq[i]*self.DM_seq[i])
        print([round(elem,2) for elem in self.cmb_seq])
        return self.GM_seq, self.DM_seq, self.cmb_seq
```

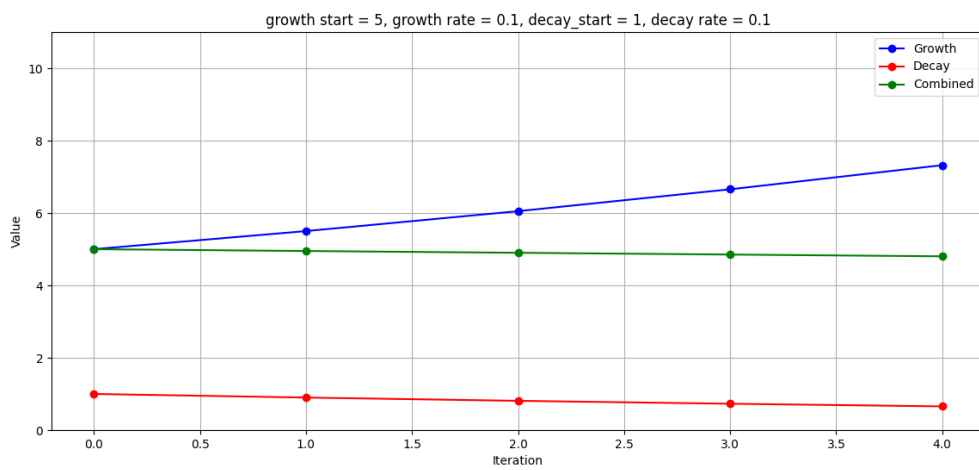
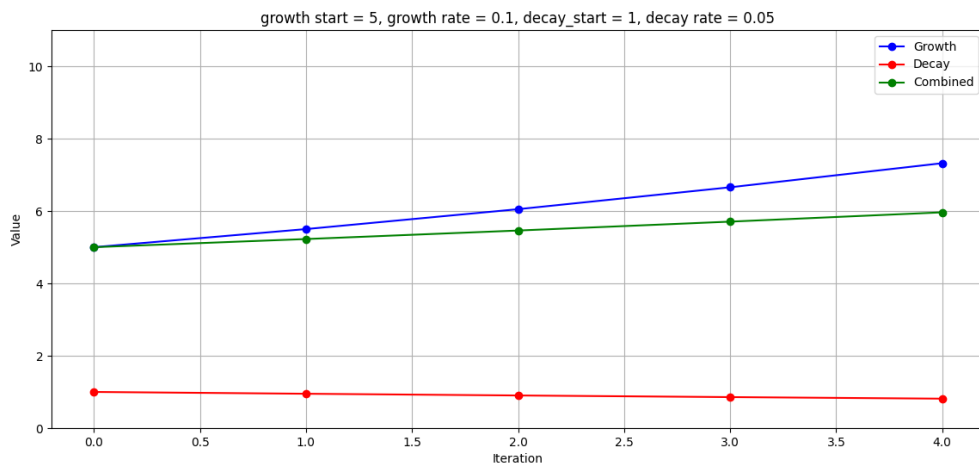
3.2 Visualization

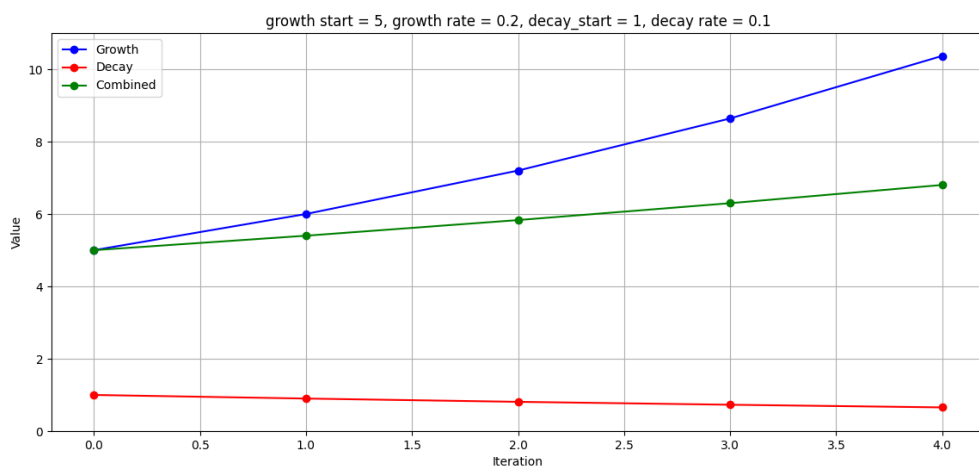
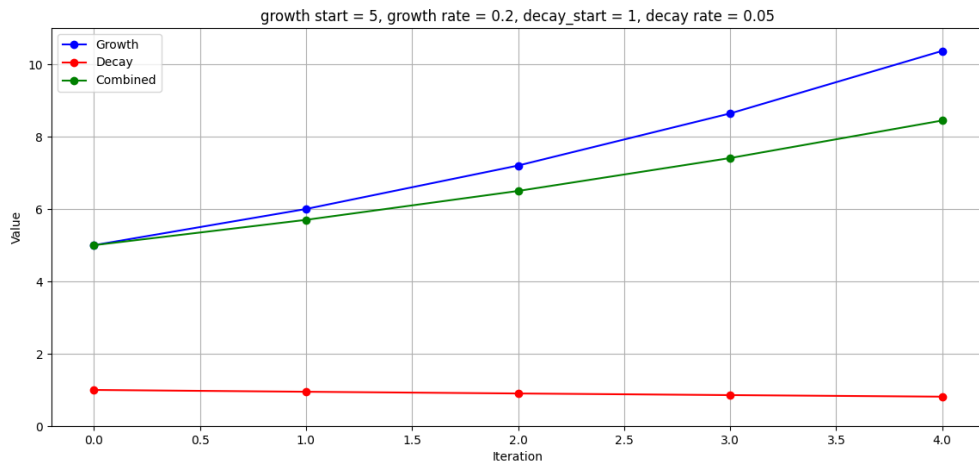
The first set of graphs have an initial growth value of 100 and an initial decay value of 1.





It is difficult to observe the decay trend in above set of graphs as the initial value of the decay is much smaller than the initial value of growth. Therefore, I have also provided a second set of graphs where the initial growth value is 5 and the initial decay value is 1.





4 Code

```
import matplotlib.pyplot as plt

class BioModel(object):
    def __init__(self, sequence):
        self.sequence = sequence
    def __len__(self):
        return len(self.sequence)
    def __iter__(self):
        self.index = -1
        return self
    def __next__(self):
        self.index += 1
        if self.index < len(self):
            return self.sequence[self.index]
        else:
            raise StopIteration
    def __eq__(self, other):
        if len(self) != len(other):
            raise ValueError('Two arrays are not equal in length!')
        else:
            count = 0
            for i in range(len(self)):
                if self.sequence[i] == other.sequence[i]:
                    count += 1
            return count

class ExponentialGrowthModel(BioModel):
    def __init__(self, start, rate):
        self.start = start
        self.rate = rate
    def __call__(self, length):
        self.sequence = [self.start]
        for i in range(length-1):
            temp = self.sequence[i]*(1+self.rate)
            self.sequence.append(temp)
        print([round(elem,2) for elem in self.sequence])
        return self.sequence

class ExponentialDecayModel(BioModel):
    def __init__(self, start, rate):
        self.start = start
        self.rate = rate
    def __call__(self, length):
        self.sequence = [self.start]
        for i in range(length-1):
            temp = self.sequence[i]*(1-self.rate)
            self.sequence.append(temp)
        print([round(elem,2) for elem in self.sequence])
        return self.sequence

class CombinedBioModel(BioModel):
    def __init__(self, growth_start, growth_rate, decay_start, decay_rate):
        self.growth_start = growth_start
        self.growth_rate = growth_rate
        self.decay_start = decay_start
        self.decay_rate = decay_rate
```

```

def __call__(self, length):
    GM = ExponentialGrowthModel(start=self.growth_start, rate=self.growth_rate)
    DM = ExponentialDecayModel(start=self.decay_start, rate=self.decay_rate)
    self.GM_seq = GM(length)
    self.DM_seq = DM(length)
    self.cmb_seq = []
    for i in range(length):
        self.cmb_seq.append(self.GM_seq[i]*self.DM_seq[i])
    print([round(elem,2) for elem in self.cmb_seq])
    return self.GM_seq, self.DM_seq, self.cmb_seq

def bonus(growth_start, decay_start):
    growth_rate = [0.1, 0.2]
    decay_rate = [0.05, 0.1]
    for i in range(2):
        for j in range(2):
            CBM = CombinedBioModel(growth_start, growth_rate[i], decay_start, decay_rate[j])
            gm_seq, dm_seq, comb_seq = CBM(length=5)
            plt.figure(figsize=(14,6))
            plt.plot(range(5), gm_seq, 'o-b')
            plt.plot(range(5), dm_seq, 'o-r')
            plt.plot(range(5), comb_seq, 'o-g')
            plt.xlabel('Iteration')
            plt.ylabel('Value')
            if growth_start == 5:
                plt.ylim((0,11))
            else:
                plt.ylim((0,220))
            plt.legend(['Growth', 'Decay', 'Combined'])
            plt.grid()
            plt.title(f'growth start = {growth_start}, growth rate = {growth_rate[i]}, decay_start = {decay_start}')
            plt.savefig(f'figure growth start = {growth_start}, growth rate = {growth_rate[i]}, d

    return

##### Task 1 (parameters from assignmnt) #####
GM = ExponentialGrowthModel(start=100, rate=0.1)
GM(length=5) # [100, 110.0, 121.0, 133.1, 146.41]
print(len(GM)) # 5
print([round(n,2) for n in GM]) # [100, 110.0, 121.0, 133.1, 146.41]

DM = ExponentialDecayModel(start=100, rate=0.2)
DM(length=5) # [100, 80.0, 64.0, 51.2, 40.96]
print(len(DM)) # 5
print([round(n,2) for n in DM]) # [100, 80.0, 64.0, 51.2, 40.96]

GM = ExponentialGrowthModel(start=100, rate=0.1)
GM(length=5) # [100, 110.0, 121.0, 133.1, 146.41]

GM2 = ExponentialGrowthModel(start=100, rate=0.2)
GM2(length=5) # [100, 120.0, 144.0, 172.8, 207.36]

print(GM == GM2) # 1

GM3 = ExponentialGrowthModel(start=100, rate=0.2)
GM3(length=3) # [100, 120.0, 144.0]

```

```

print(GM == GM3) # ValueError: Two arrays are not equal in length!
#####

##### Task 2 (parameters from assignmnt) #####
GM = ExponentialGrowthModel(start=50,rate=0.2)
GM(length=5) # [50, 60.0, 72.0, 86.4, 103.68]
print(len(GM)) # 5
print([round(n,2) for n in GM]) # [50, 60.0, 72.0, 86.4, 103.68]

DM = ExponentialDecayModel(start=50,rate=0.3)
DM(length=5) # [50, 35.0, 24.5, 17.15, 12.0]
print(len(DM)) # 5
print([round(n,2) for n in DM]) # [50, 35.0, 24.5, 17.15, 12.0]

GM = ExponentialGrowthModel(start=50,rate=0.2)
GM(length=5) # [50, 60.0, 72.0, 86.4, 103.68]

GM2 = ExponentialGrowthModel(start=50,rate=0.3)
GM2(length=5) # [50, 65.0, 84.5, 109.85, 142.81]

print(GM == GM2) # 1

GM3 = ExponentialGrowthModel(start=50,rate=0.2)
GM3(length=3) # [50, 60.0, 72.0]

print(GM == GM3) # ValueError: Two arrays are not equal in length!
#####

##### Bonus Task #####
CBM = CombinedBioModel(growth_start=100,growth_rate=0.1,decay_start=1.0,decay_rate=0.05)
comb_seq = CBM(length=5)

bonus(5,1)
bonus(100,1)

#####

```
